Introduction à l'économétrie

Chapitre 21 : Mise en œuvre sur R et Python

Jaouad Madkour

Faculté d'Économie et de Gestion de Tanger · 23 octobre 2026

Plan du chapitre

Lire une sortie

Sur R

Sur Python

Ce que le logiciel ne fait pas

S'entraîner

Ce qu'il faut retenir

Lire une sortie

Neuf nombres, et vous savez tous les lire

	Estimate	Std. Error	t value	Pr(> |t|)
(Intercept)	3320,42	158,85	20,90	< 2e-16
anciennete	270,27	11,72	23,06	< 2e-16
diplomeBac+2	1099,57	158,59	6,93	1,3e-11
diplomeBac+5	3336,99	184,25	18,11	< 2e-16
Residual standard error: 1561 on 496 degrees of freedom				
Multiple R-squared: 0,6418 Adjusted: 0,6396				
F-statistic: 296,2 on 3 and 496 DF, p-value: < 2,2e-16				

Neuf nombres, et vous savez tous les lire

Estimate — les $\widehat{\beta}_j$, chapitres 3 et 7.

Std. Error — $\widehat{\sigma}(\widehat{\beta}_j)$, chapitre 5.

t value — le rapport des deux, chapitre 5.

Pr — la p-value, chapitre 14 de l'inférence.

Residual standard error — $\widehat{\sigma}_\varepsilon$, l'écart-type des résidus.

degrees of freedom — $n - k - 1$, un degré perdu par paramètre estimé, chapitre 4 de l'inférence.

R-squared — SCE/SCT, chapitre 4.

F-statistic — le test global, chapitre 9.

Ce que la sortie ne dit pas

Rien sur l'hétéroscédasticité, rien sur les variables

C'est l'aboutissement de quatre cours.

La moyenne et la variance viennent de la descriptive ; la loi de Student et le théorème central limite, des probabilités ; l'erreur-type, le test et la p-value, de l'inférence ; la décomposition et le F, de l'analyse de la variance. **L'économétrie n'a rien ajouté d'autre qu'une question économique.**

La sortie complète de summary

```r
sal <- read.csv("salaires.csv", stringsAsFactors = TRUE)
summary(lm(salaire ~ anciennete + diplome, data = sal))
#
# Call:
# lm(formula = salaire ~ anciennete + diplome, data = sal)
#
# Residuals:
#     Min      1Q  Median      3Q     Max
# -3541.7 -1056.2  -192.4  1048.0  6269.9
#
# Coefficients:
#              Estimate Std. Error t value Pr(>|t|)
# (Intercept)   3320.42     158.85  20.902  < 2e-16
# anciennete     270.27      11.72  23.059  < 2e-16
# diplomeBac+2  1099.57     158.59   6.933 1.29e-11
# diplomeBac+5  3336.99     184.25  18.111  < 2e-16
#
# Residual standard error: 1561 on 496 degrees of freedom
# Multiple R-squared:  0.6418, Adjusted R-squared:  0.6396
# F-statistic: 296.2 on 3 and 496 DF,  p-value: < 2.2e-16
```

Chaque ligne, et le chapitre qui l'a établie

De haut en bas

Ligne de la sortie	Ce que c'est	Ch.
`Residuals`	quartiles des e_i	2, 12
`Estimate`	les $\widehat{\beta}_j$	3, 7
`Std. Error`	$\widehat{\sigma}(\widehat{\beta}_j)$, sous H3 et H4	5, 11
`t value`, `Pr(>|t|)`	test de $\beta_j = 0$	5
`Residual standard error`	$\widehat{\sigma}_\varepsilon$	4
`R-squared`	R^2, R^2 ajusté	4, 8
`F-statistic`	test global	9

Ce que la sortie ne dit pas : ses écarts-types sont ordinaires, valides sous H3 et H4 seulement.

Sur R

```r
sal <- read.csv("salaires.csv", stringsAsFactors = TRUE)
sal$diplome <- relevel(sal$diplome, ref = "Bac")  # la référence
m1 <- lm(salaire ~ anciennete, data = sal)
m2 <- update(m1, . ~ . + diplome)
round(confint(m2), 2)                # Student à 496 ddl
#                2.5 %  97.5 %
# (Intercept)  3008.31 3632.53
# anciennete    247.24  293.30
# diplomeBac+2  787.98 1411.16
# diplomeBac+5 2974.98 3699.00
```

```r
suppressPackageStartupMessages({library(lmtest); library(car)})
sal <- read.csv("salaires.csv", stringsAsFactors = TRUE)
m2 <- lm(salaire ~ anciennete + diplome, data = sal)
m4 <- update(m2, . ~ . + sexe + region)
bptest(m2)$statistic                    # Breusch-Pagan
#       BP
# 57.59655
round(vif(m4)[, 1], 2)                  # VIF généralisés
# anciennete    diplome       sexe     region
#       1.01       1.01       1.01       1.02
shapiro.test(residuals(m2))$p.value  # normalité
# [1] 3.638585e-06
sum(cooks.distance(m2) > 4 / nrow(sal))   # points à examiner
# [1] 33
```

Un VIF par facteur

`car::vif` regroupe les indicatrices d'un même facteur : la région a un VIF généralisé de $1{,}02$, alors que ses indicatrices ont, une à une, des VIF voisins de 2 — la liaison entre elles est due à la construction, pas aux données.

Shapiro plutôt que Jarque-Bera

R propose d'emblée le test de Shapiro-Wilk, plus puissant que Jarque-Bera : $p = 3{,}6 \times 10^{-6}$, même conclusion qu'au chapitre 17 — et la même absence de conséquence sur 500 observations.

plot(m2)

`par(mfrow = c(2, 2)) ; plot(m2)` produit les quatre graphiques du chapitre 12 en une instruction : résidus contre valeurs ajustées, quantile-quantile, échelle-position, résidus contre levier.

Quatre secondes, et l'essentiel du diagnostic est fait

La figure du chapitre 12 a été produite exactement ainsi. Le premier panneau montre l'éventail de l'hétéroscédasticité, le deuxième le décrochage des hauts salaires : deux chapitres entiers, annoncés par une image.

```r
suppressPackageStartupMessages(library(lmtest))
library(sandwich)
sal <- read.csv("salaires.csv", stringsAsFactors = TRUE)
m2 <- lm(salaire ~ anciennete + diplome, data = sal)
coeftest(m2, vcov = vcovHC(m2, type = "HC1"))
#
# t test of coefficients:
#
#               Estimate Std. Error t value  Pr(>|t|)
# (Intercept)  3320.421    144.365 23.0002 < 2.2e-16
# anciennete    270.269     13.003 20.7844 < 2.2e-16
# diplomeBac+2 1099.570    138.431  7.9431 1.332e-14
# diplomeBac+5 3336.992    226.280 14.7472 < 2.2e-16
```

```r
suppressPackageStartupMessages({library(lmtest); library(car)})
men <- read.csv("menages.csv", stringsAsFactors = TRUE)
mk <- lm(consommation ~ revenu, data = men)
mk2 <- lm(consommation ~ revenu + taille + milieu, data = men)
linearHypothesis(mk, "revenu = 1")$F[2]       # Keynes : F = t^2
# [1] 426.9366
V <- sandwich::vcovCL(mk2, cluster = ~ quartier)
round(unclass(coeftest(mk2, vcov = V))[, 1:3], 3)
#              Estimate Std. Error t value
# (Intercept)   497.336     97.028   5.126
# revenu          0.705      0.023  30.349
# taille         70.669     12.291   5.750
# milieuUrbain  -99.440    101.917  -0.976
```

Deux lignes concluent deux chapitres

linearHypothesis et vcovCL

`linearHypothesis` teste n'importe quelle contrainte linéaire — ici la restriction que la théorie de Keynes impose : $F = 426{,}9 = (-20{,}66)^2$. **La démarche des quatre étapes s'achève sur une commande.**

`vcovCL` corrige le regroupement : l'écart-type du milieu urbain passe de $55{,}69$ à $101{,}92$, parce que cette variable est **constante dans le quartier**.

Ce test-là utilise les écarts-types ordinaires

Sans autre précision, `linearHypothesis` prend la variance des MCO. Pour un test robuste aux grappes, on lui passe `vcov. = vcovCL(...)` : le t de Keynes tombe alors à $-14{,}3$, comme au chapitre 20.

Sur Python

```
import pandas as pd
import statsmodels.formula.api as smf
sal = pd.read_csv("salaires.csv")
f = "salaire ~ anciennete + C(diplome, Treatment('Bac'))"
m2 = smf.ols(f, data=sal).fit()
t = pd.DataFrame({"coef": m2.params, "se": m2.bse,
                  "t": m2.tvalues})
t.index = ["const", "Bac+2", "Bac+5", "anciennete"]
print(t.round(2))
#                coef      se      t
# const       3320.42  158.85  20.90
# Bac+2       1099.57  158.59   6.93
# Bac+5       3336.99  184.25  18.11
# anciennete   270.27   11.72  23.06
```

Tester et corriger

```
import pandas as pd
import statsmodels.formula.api as smf
import statsmodels.stats.api as sms
sal = pd.read_csv("salaires.csv")
m1 = smf.ols("salaire ~ anciennete", data=sal).fit()
m2 = smf.ols("salaire ~ anciennete + diplome", data=sal).fit()
F, p, q = m2.compare_f_test(m1)       # Fisher partiel
print(round(F, 2), f"{p:.1e}", q)
# 164.01 2.1e-55 2.0
bp = sms.het_breuschpagan(m2.resid, m2.model.exog)
print(round(bp[0], 2), f"{bp[1]:.1e}")
# 57.6 1.9e-12
```

R et Python, ligne à ligne

R	Python
`lm`	`smf.ols(...).fit()`
`relevel`	`C(var, Treatment('ref'))`
`anova(m1, m2)`	`m2.compare_f_test(m1)`
`bptest`	`het_breuschpagan`
`vcovHC(type = "HC1")`	`fit(cov_type="HC1")`
`vcovCL`	`cov_type="cluster", cov_kwds={...}`
`vif`	`variance_inflation_factor`
`linearHypothesis`	`f_test`, `t_test`
`cooks.distance`	`get_influence().cooks_distance`

La régression finale de Mincer

```
import numpy as np, pandas as pd
import statsmodels.formula.api as smf
sal = pd.read_csv("salaires.csv")
sal["lsal"] = np.log(sal.salaire)
f = "lsal ~ anciennete + C(diplome, Treatment('Bac'))" \
    " + C(region, Treatment('Autres'))"
mincer = smf.ols(f, data=sal).fit(cov_type="HC1")
b = mincer.params
print(b[["anciennete"]].round(4).tolist(), round(b.iloc[2], 4))
# [0.0345] 0.4353
print(round(mincer.rsquared, 3))
# 0.654
print((100 * (np.exp(b) - 1)).round(1).tolist()[1:])   # exacts
# [19.6, 54.5, 18.0, 14.8, 6.1, 3.5]
```

Ce que le logiciel ne fait pas

Trois choses, et ce sont les trois qui comptent

Il ne choisit pas les variables

La spécification vient de la théorie économique. Aucune procédure automatique ne remplace la question : *quelles variables devraient figurer dans ce modèle, et lesquelles manquent ?*

Il ne signale pas les variables omises, ni l'endogénéité

La sortie reste parfaitement propre. Le R^2 est honorable, les t écrasants, les résidus impeccables — **et le coefficient peut être faux**.

C'est le chapitre 16 et le chapitre 19 réunis, et c'est ce qui distingue un économètre d'un utilisateur de logiciel.

Et il ne choisit pas les écarts-types

Le défaut est le plus fragile

`summary` et `fit()` donnent par défaut les écarts-types **ordinaires** — ceux qui supposent H3 et H4. Sur données individuelles, ce sont presque toujours les mauvais.

Choisir HC1 ou la grappe est une décision de l'économètre, à justifier par la structure des données : individuelles, ou tirées par groupes.

Une checklist en cinq lignes

1. La variable expliquée a-t-elle la bonne forme — niveau ou logarithme ?
2. La référence de chaque facteur est-elle celle voulue ?
3. Les écarts-types sont-ils robustes, ou de grappe s'il le faut ?
4. Les quatre graphiques ont-ils été regardés ?
5. Les limites sont-elles écrites ?

S’entraîner

Exercice 1 : relire une sortie

Une ligne de summary

Une sortie R contient la ligne `anciennete  270.27  11.72  23.059  < 2e-16`, avec `496 degrees of freedom`.

1. Retrouver le t à partir des deux premiers nombres.
2. Construire l'intervalle de confiance à $95\,\%$ ($t_{496} = 1{,}965$) et le comparer à `confint`.
3. Que deviendrait le t avec l'écart-type robuste $13{,}00$?

Exercice 1 : solution

Tout se retrouve à la main

1. $270{,}27 / 11{,}72 = 23{,}06$.
2. $270{,}27 \pm 1{,}965 \times 11{,}72 = [247{,}24\,;\,293{,}30]$: exactement la ligne de `confint`.
3. $270{,}27 / 13{,}00 = 20{,}79$ — la valeur de `coeftest` avec `vcovHC`. La conclusion ne change pas ; sa base, si.

De R vers Python

Écrire en Python l'équivalent de :

```
mk2 <- lm(consommation ~ revenu + taille + milieu, data = men)
coeftest(mk2, vcov = vcovCL(mk2, cluster = ~ quartier))
```

Une option au lieu d'une fonction

```
f = "consommation ~ revenu + taille + milieu"
mk2 = smf.ols(f, data=men).fit(
    cov_type="cluster", cov_kwds={"groups": men.quartier})
print(mk2.summary())
```

On retrouve $101{,}92$ pour le milieu urbain : les deux logiciels appliquent la même correction de petit nombre de grappes.

Trois scripts fautifs

Qu'est-ce qui ne va pas ?

1. `lm(log(depenses_pub) ~ ventes, data = d)`, alors que $40\,\%$ des entreprises n'ont aucune dépense publicitaire.
2. `lm(salaire ~ anciennete + region, data = sal)` lu en disant « Casablanca rapporte $1\,174$ dirhams de plus que Rabat ».
3. `summary(lm(salaire ~ diplome, data = sal))` publié tel quel dans un mémoire sur 500 salariés.

Exercice 3 : solution

Le zéro, la référence, les écarts-types

1. $\ln 0$ n'existe pas : R écarte sans prévenir les $40\,\%$ d'entreprises sans publicité — l'échantillon a changé. Vérifier `nobs`.
2. Les coefficients régionaux se lisent par rapport à la **référence**, ici « Autres » ; la comparaison avec Rabat demande la différence des deux coefficients, et un test.
3. Écarts-types **ordinaires** sur données individuelles hétéroscédastiques : il faut des écarts-types robustes — et se demander quelles variables manquent.

Ce qu’il faut retenir

Le résumé du chapitre

Six points

1. Chaque nombre de la sortie de `lm` ou `smf.ols` a été calculé à la main dans le cours.
2. `relevel` ou `Treatment` fixent la **référence** des indicatrices : à vérifier systématiquement.
3. `plot(m)` donne les **quatre graphiques** de diagnostic en une instruction.
4. `vcovHC` et `vcovCL`, ou `cov_type`, corrigent les écarts-types sans toucher aux coefficients.
5. `linearHypothesis` et `f_test` testent une contrainte **économique**.
6. Le logiciel ne choisit ni les variables, ni la forme, ni les écarts-types, et ne signale jamais l'**endogénéité**.

La fin du parcours

Quatre cours, un même fichier de cinq cents salariés : décrit, échantillonné, puis expliqué. **Une seule discipline, enseignée en quatre fois** — et un tableau de régression, que vous savez maintenant lire, en est l'aboutissement.